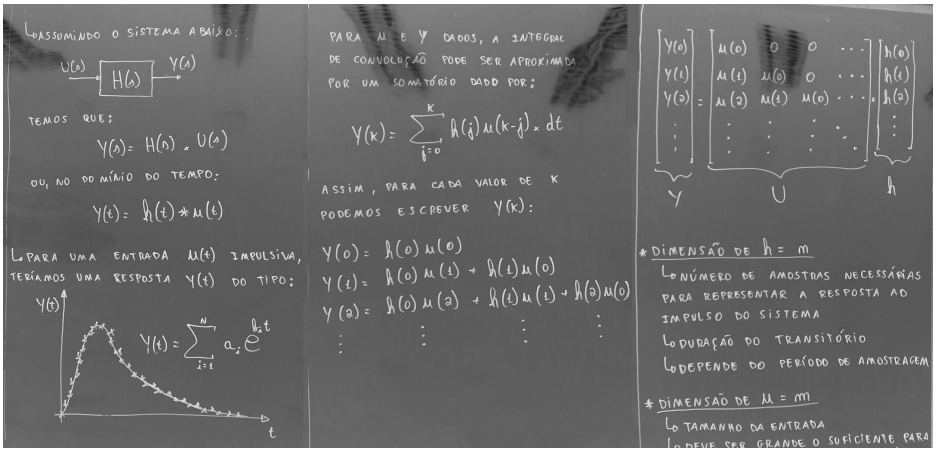
Notas de Aula de
MODELAGEM E SIMULAÇÃO



uma abordagem experimental para obtenção de modelos determinísticos e simulação numérica

Prof. Lucas Molina
lmolina@academico.ufs.br

TÓPICO 5 : Aspectos de convergência e de implementação do GN

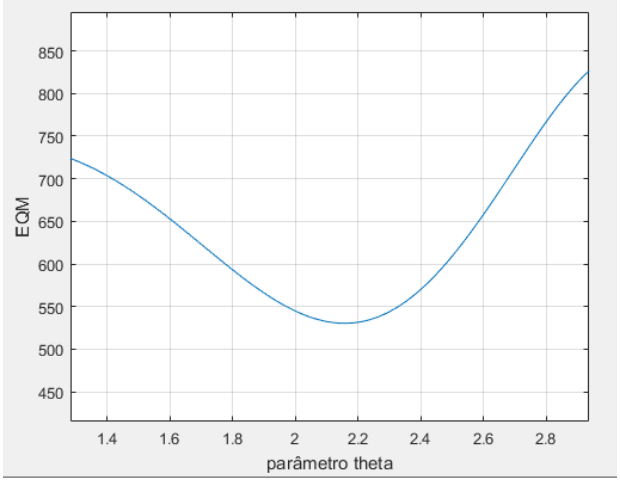
- Como visto no tópico anterior, o GN é um método recursivo baseado em uma sequência de aproximações lineares. Essas aproximações são feitas em torno de um ponto de operação e a sua validade está restrita às proximidades desse ponto.
- Adicionalmente, vimos que dado um conjunto de amostras de entrada e saída o método é determinístico, ou seja, se executado várias vezes obteremos sempre o

mesmo resultado. Esse aspecto indica a importância da inicialização dos parâmetros, ou seja, a escolha do primeiro ponto de operação.

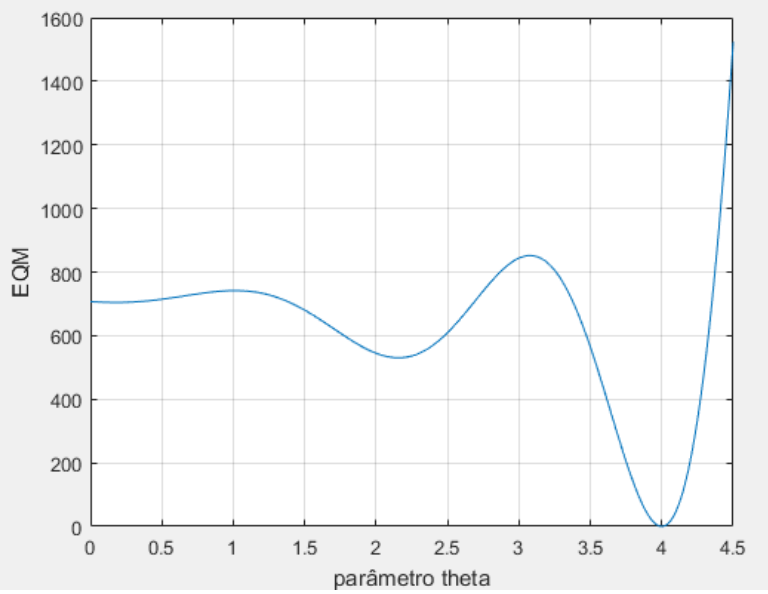
- Esses e outros aspectos de implementação e convergência serão melhor discutidos neste tópico.

Inicialização dos Parâmetros:

- Quando temos uma função com relação linear entre os parâmetros, a função a ser otimizada pelo MMQ é sempre uma forma quadrática, ou seja, uma função que possui um único mínimo chamado de mínimo global.
- Quando eliminamos essa restrição, como é o caso do GN, a função a ser minimizada, ou seja, o somatório dos erros ao quadrado, não é mais garantidamente uma função com um único mínimo.
- Os mínimos dessa função são chamados de **mínimo local**, se existirem outros mínimos menores que ele ou de **mínimo global** se o seu valor for o menor possível para a função.
- Para entender melhor essa questão, considere que temos a seguinte função a minimizar:
    - $y = x^{\theta}$
    - Em que o valor de **θ** é o parâmetro a determinar.
- Se o gráfico abaixo fosse apresentado a você, no qual o **eixo x** representa valores da variável **θ** e o **eixo y** representa o erro ao quadrático médio em relação às amostras coletadas e o modelo criado com o valor de θ (**EQM**), qual o valor de **θ** que você escolheria que minimiza o EQM?



- Claramente, qualquer pessoa com alguma noção e sanidade escolheria um valor em torno de 2,18734.
- Como a função $y = x^{\theta}$ não pode ser escrita como $y = \theta f(x)$, não podemos utilizar o MMQ. Mas podemos utilizar o GN.
- O GN, se tivesse seus parâmetros iniciados dentro da janela ilustrada na figura, certamente encontraria a mesma solução que nós encontramos por observação, só que sem precisar calcular o EQM para todos os valores de θ.
- A essa janela damos o nome de **bacia de atração** ou **bacia de convergência**.
- Agora vamos rever o mesmo gráfico, só que considerando uma janela maior, e refazer a análise.



- Para essa nova visualização, qual seria a sua escolha de **θ** para minimizar o EQM?
- Mais uma vez, fazendo uso de mera observação, nós somos obrigados a escolher o valor **θ = 4.**
- Mas calma!
    - Esse gráfico é o mesmo gráfico anterior só que com uma janela de observação maior.
    - Se antes, quando o GN tivesse uma inicialização de **θ** na bacia de convergência do valor **2,18734** ele encontraria esse ponto, agora ele fará também o mesmo!

    - Se isso está correto, então se o método de Gauss Newton for utilizado nesse problema e a variável **θ** for iniciada com valor igual a **2,5** (por exemplo) o **GN achará uma solução errada?**
        - **SIM!** Ele encontrará uma solução errada! Essa solução errada é chamada de **mínimo local** por ser um mínimo se considerarmos apenas o seu entorno, mas não é fato o menor valor possível para o EQM.
    - Por outro lado, se o GN for inicializado em outra **bacia de convergência** cujo único mínimo é o valor **θ = 4** ele deve encontrar a solução correta, nosso **mínimo global**.
- O que podemos tirar deste exemplo ilustrativo?
    - **A depender da inicialização, o método pode ou não convergir para a solução desejada.**
- Como podemos garantir que a inicialização está na bacia de convergência do mínimo global?
    - De forma geral, não podemos.
    - Existem técnicas que são aplicadas caso a caso, que devem ser repetidas para cada nova função, que ajudam na determinação dessa bacia. Mas de maneira geral, considerando funções quaisquer, não há muitas formas analíticas para isso.
    - Tendo em vista que a determinação analítica de bacias de convergência extrapola o alcance dessa disciplina, utilizaremos uma abordagem numérica para mitigar esse problema: **Inicialização aleatória**.
- As vantagens de uma inicialização aleatória são exploradas em muitos métodos de ajuste recursivo de parâmetros como o Gauss Newton, DLS, Redes Neurais, Algoritmos Genéticos, entre outros.
- De forma simples, a ideia tem por objetivo transformar um método determinístico de ajuste de parâmetros em um método estocástico, quando se considera a inicialização como parte do método. Além disso, quando estamos trabalhando com problemas com muitos parâmetros, escolher manualmente cada um deles pode ser um trabalho tedioso, especialmente quando se considera que no caso de não convergência você precisará escolher um novo ponto de partida para tentar novamente encontrar uma solução.

- Considerando que estamos trabalhando agora com uma inicialização aleatória: **Qual a distribuição de probabilidade que vamos usar para cada um dos parâmetros?**
    - Uma técnica simples é definir todos os parâmetros utilizando uma distribuição uniforme entre -1 e 1 ou entre -0,1 e 0,1. Essa solução costuma dar bons resultados em uma boa variedade de problemas.
    - Outra possibilidade é uma escolha mais rebuscada para essas distribuições de probabilidade, escolhendo uma distribuição uniforme específica para cada parâmetro.
        - Nesse caso, a escolha das distribuições devem levar em consideração algumas características de cada função. Em geral, a distribuição deve ser escolhida de modo a:
            - Minimizar o Δy
            - Minimizar as derivadas parciais da matriz Jacobiana
    - Seja qual for a distribuição de probabilidade, como não sabemos a bacia de convergência do mínimo global, **não há garantia de convergência para a solução global.**
- Exercício 1: Vamos verificar experimentalmente os dados apresentados aqui. Gere um conjunto de pontos para a função utilizada como exemplo aqui ($y=x^4$), para um vetor de entradas x (no Octave -> x = [-3:0.4:3]') sem adicionar ruído em y. Usando um α = 0.1, utilize o GN para encontrar a solução do problema de duas maneiras:
    - inicialização **θ = 2,5**
    - inicialização **θ = 4,5**

  Verifique se a resposta encontrada está de acordo com o apresentado aqui.

Atualização dos Parâmetros:

- Na explicação do GN falamos sobre a importância de termos passos pequenos para evitar se afastar da região de validade do modelo linearizado.
- Para contornar esse problema definimos duas abordagens:
    - Amortecimento linear (α)
    - Saturação do passo em um valor máximo

- O amortecimento e a saturação são parâmetros que dependem de cada problema. Assim, precisaremos sempre fazer ajuste nesses parâmetros.
- **Amortecimento linear (α):**
    - Valores entre 0 e 1
    - Quanto menor o valor de α, mais lento é o algoritmo, porém, os passos pequenos gerados pelo valor baixo ajudam para que a convergência seja mais estável
    - Quanto mais próximo de 1 é o valor de α, mais rápida é a convergência porém mais instável, correndo o risco de fazer o valor dos parâmetros divergir
    - Eventualmente um passo grande pode ajudar o GN a escapar de um mínimo local, mas a desvantagem da instabilidade normalmente é mais crítica sendo preferível valores menores.
    - Apesar de que com valores menores o sistema é mais estável, a velocidade muito baixa pode fazer com o sistema evolua para a solução muito lentamente ou mesmo não consiga se aproximar da solução por conta do passo muito pequeno (tendendo a zero) perto do mínimo global.
    - Em geral, deve-se buscar uma solução de compromisso e ajustar o parâmetro com base na observação da velocidade de convergência do método.
- Exercício 2: Repita o exercício anterior agora utilizando um amortecimento de 0,8. O que você observou de diferente?
- **Saturação:**
    - Muitas vezes, o valor do amortecimento não é adequado para garantir um passo pequeno para o vetor $\Delta\theta$. Nesse caso, a saturação é uma saída para limitar o tamanho do passo.
    - Como a saturação é uma operação não linear, ela deve ser usada preferencialmente depois do amortecimento.
    - A ideia é simples. Define-se um módulo máximo para o vetor $\Delta\theta$, chamado aqui de **Dmax.** Se o módulo do valor do passo depois do amortecimento for maior do que Dmax, o módulo é saturado em Dmax, mantendo o mesmo sinal.

- Uma sugestão de código para essa etapa é ilustrado abaixo:

```
...
dth = inv(J'*J)*J'*dy;

dth = alfa*dth;                          % amortecimento primeiro

if (norm(dth) > Dmax)                    % se a norma for maior que o limite
    dth = Dmax*dth/norm(dth)             % satura a norma
end

th = th + dth;                           % atualiza o parâmetro com o resultado

...
```

- Uma outra alternativa é saturar elemento por elemento do vetor $\Delta\theta$.
- Uma sugestão de código para essa outra forma de saturar é ilustrado na figura abaixo:

```
...
dth = inv(J'*J)*J'*dy;

dth = alfa*dth;                          % amortecimento primeiro

for j=1:length(dth)                      % para cada um dos elementos
    if (abs(dth(j)) > Dmax)              % se o elemento for maior que o limite
        dth(j) = Dmax*dth(j)/abs(dth(j))     % satura o módulo do elemento
    end
end

th = th + dth;                           % atualiza o parâmetro com o resultado

...
```

- Assim como no caso do amortecimento linear, a suturação não é capaz de garantir a convergência para um caso qualquer.
- Esse parâmetro deve também ser utilizado para fazer ajuste na convergência

Critério de Parada:

- O critério de parada não tem qualquer interferência na execução do método.
- É possível executar o código e quando você achar que o resultado já tá bom visualmente, interromper manualmente a execução (ctrl+c).
- No entanto, se quiser automatizar essa parada do algoritmo, basta monitorar ao final de cada iteração o valor do módulo de $\Delta\theta$. Quando esse valor atingir um valor próximo o suficiente de zero você interrompe o loop (comando **break**).
- Uma outra possibilidade é monitorar se algum elemento do vetor $\Delta\theta$ virou **NaN**.
    - **NaN** significa ***Not a Number*** e é utilizado pelo Octave/Matlab para dizer que uma operação retornou um resultado não numérico.
    - Quando isso acontece, não adianta continuar executando o GN que ele não irá convergir para lugar algum. Na verdade, na iteração seguinte possivelmente todos os parâmetros calculados se tornarão NaN.
    - Assim, uma outra coisa interessante para incluir no critério de parada é verificar se um elemento qualquer do vetor $\Delta\theta$ se tornou NaN. Caso isso aconteça, o loop deve ser interrompido também (comando **break**).
    - Para saber se um número é NaN, utilize a função **isnan.**

Visualização em tempo real do ajuste:

- Para acompanhar o desempenho do algoritmo, é interessante utilizar uma visualização em tempo real à medida que um novo vetor de parâmetro é calculado (atualização do ponto de operação)
- No entanto, como a prioridade do plot no Octave/Matlab é baixa, é preciso forçar o programa a parar a execução e se dedicar ao plot. Do contrário, a movimentação do modelo a cada iteração não será visível na figura.
- Para fazer isso, precisamos a cada iteração atualizar o gráfico. No Matlab, fazemos normalmente utilizando o **drawnow** em conjunto com o **hold on** e **hold off** e funciona de forma eficiente, mas no Octave o desempenho deixa a desejar.
- Buscando uma forma de uniformizar e que seja útil tb no Octave, podemos utilizar o código abaixo para plotar o modelo a cada iteração dentro do loop. Lembrando que isso é só uma sugestão, você pode fazer como achar melhor. Nessa abordagem

utilizaremos o **drawnow** para forçar o plot e o **set** para atualizar o gráfico apenas do modelo.

```
...
% sendo y os dados coletados e ym os dados do modelo atual

p1 = plot(x,y,'*')
hold on
p2 = plot(x,ym,'r')


for j=1:10000                          % loop do GN

  ...
  ...                                    % atualização do modelo ym
  ...

   set(p2,'YData',real(ym))
   drawnow

end
```

Importância desse tópico para outros métodos iterativos:

- Neste tópico nós discutimos alguns aspectos de convergência e de implementação do método de Gauss Newton.
- É importante salientar que os temas discutidos aqui, apesar de usar o GN como exemplo, são temas importantes para a grande maioria das técnicas iterativas, portanto, essas informações podem ser extrapoladas para outras metodologias como DLS que estudaremos em breve.

Funções Genéricas com relação não linear entre os parâmetros:

- Agora que já discutimos e implementamos muitos aspectos do GN está na hora de usar um pouco a nossa ferramenta.
- Assim como foi no caso do MMQ, no GN também temos bases genéricas de funções, ou seja, funções genéricas com a capacidade de representar "funções quaisquer".
- No MMQ utilizamos duas bases principais:
    - Polinomial
    - Série de Fourier
- No GN temos a possibilidade de utilizar funções mais complexas. Existem algumas bases de interesse e que são úteis para uma grande variedade de problemas. As duas principais para os nossos métodos são:
    - **Somatório de sigmóides**:
        - Uma entrada:

$$y = \theta_1 + \theta_2 tanh(\theta_3 x + \theta_4) + \theta_5 tanh(\theta_6 x + \theta_7) + \theta_8 tanh(\theta_9 x + \theta_{10}) + \ldots$$

        - Duas entradas (x_1 e x_2):

$$y = \theta_1 + \theta_2 tanh(\theta_3 x_1 + \theta_4 x_2 + \theta_5) + \theta_6 tanh(\theta_7 x_1 + \theta_8 x_2 + \theta_9) + \theta_{10} tanh(\theta_{11} x_1 + \theta_{12} x_2 + \theta_{13}) + \ldots$$

    - **Somatório de exponenciais complexas**:

$$y = \theta_1 + \theta_2 e^{\theta_3 x} + \theta_4 e^{\theta_5 x} + \theta_6 e^{\theta_7 x} + \ldots$$

- **O somatório de sigmóides** é possivelmente a base mais utilizada para ajuste de funções. Bastante utilizada por sua capacidade de representação demonstrada no **Teorema da Aproximação Universal**, essa base é a mais utilizada em técnicas com Redes Neurais, sendo inclusive a principal escolha para a **Função de Ativação** (na representação de neurônios).

- **O somatório de exponenciais complexas** é muito interessante para o estudo e modelagem de sistemas lineares. Como visto em Análise de Sistemas Lineares, a resposta de um sistema dinâmico a uma entrada qualquer trará características da entrada e características do sistema em si. A parte referente ao sistema, que pode ser sintetizada na resposta ao impulso, pode ser representada por um somatório de exponenciais complexas. Daí a importância dessa base.
- Uma outra base muito importante é o Somatório de Gaussianas. No entanto, o ajuste de parâmetros para essa base não é eficiente para técnicas baseadas em derivadas parciais como é o caso do GN, DLS e da própria Rede Neural quando usada com Backpropagation para o ajuste de pesos.
- Exercício 3: Utilize o GN para tentar encontrar um modelo para cada um dos conjuntos de dados, pares de vetores (x,y), presentes nos arquivos “dados_GNa.mat” e “dados_GNb.mat” disponibilizados.